

Bluetooth Slave Mode

Bluetooth Slave Mode

1. Bluetooth Slave Overview
2. Core Concepts
 - 2.1 GAP — Generic Access Profile
 - 2.2 GATT — Generic Attribute Profile
 - 2.3 Notify vs Write
 - 2.4 NimBLE-Arduino Library Architecture
 - 2.5 Notify vs Indicate Implementation Differences in Arduino
3. Hardware Dependencies
 - 3.1 ESP32-S3 BLE Hardware Specifications
 - 3.2 Required Libraries
4. Project Architecture and Flow
5. Source Code Analysis
 - 5.1 Header Files and Global Variables
 - 5.2 Receiving Master Data — WriteCallbacks
 - 5.3 Connect/Disconnect Event Callbacks — ServerCallbacks
 - 5.4 FreeRTOS Push Task — sendTask
 - 5.5 setup() — Initialize BLE Slave
 - 5.6 loop() — Empty Loop
6. Operation Procedures
 - 6.1 Build and Flash
 - 6.2 Method 1: Phone Bluetooth Debug
 - 6.3 Method 2: Dual-Board Communication Test (Requires Two Boards)
 - 6.4 Expected Output — Phone Debug
 - 6.5 Expected Output — Dual-Board Communication
 - 6.6 Log Field Description
 - 6.7 Edge Cases
7. FAQ

1. Bluetooth Slave Overview

BLE (Bluetooth Low Energy) is a short-range wireless communication protocol designed for low power consumption and low bandwidth scenarios. Compared to Classic Bluetooth, BLE connections are faster (millisecond-level) and consume less power (coin batteries can run for months).

Importance of BLE Slave Tutorial: BLE Slave (Peripheral) is the most common role for IoT devices — sensor nodes, wearable devices, smart home peripherals, etc. Most scenarios run as slaves. Mastering slave programming means you can build smart devices that can be discovered and connected by phones, tablets, or gateways. This is a core skill for embedded BLE development.

This experiment uses the NimBLE protocol stack (saves ~50% RAM compared to Arduino's default Bluedroid), creating a FreeRTOS task that pushes data to the master every 2 seconds, while also receiving data written by the master.

After completing this tutorial, you will have: An ESP32-S3 development board broadcasting with device name `ESP-IDF`. Phones or another ESP32-S3 can scan, connect, receive pushed data every 2 seconds, and send arbitrary text commands to the slave — a complete bidirectional BLE communication slave.

Typical Application Scenarios:

Application Type	Example
Sensor Slave	Thermometer broadcasts and pushes readings
Wearable Device	Fitness band pushes heart rate data to phone
Beacon	iBeacon location broadcast

BLE Role System:

Role	GAP Name	Responsibility
Slave	Peripheral	Broadcasts presence, accepts connections, provides GATT services for master access
Master	Central	Actively scans nearby devices, initiates connections, discovers services and characteristics, reads/writes data

2. Core Concepts

2.1 GAP — Generic Access Profile

GAP (Generic Access Profile) defines how BLE devices are **scanned and connected**.

GAP operations for this slave:

Operation	Arduino API	Description
Advertising	<code>NimBLEDevice::getAdvertising()->start()</code>	Periodically sends advertising packets containing device name and service UUID
Connection Response	<code>ServerCallbacks::onConnect()</code>	Accepts master connection request, updates connection status
Re-advertising on Disconnect	<code>ServerCallbacks::onDisconnect()</code> → <code>NimBLEDevice::startAdvertising()</code>	Automatically restarts advertising after disconnection

Advertising Packet Structure

```
Advertising Packet
├─ Flags: General discoverable + BR/EDR not supported
├─ Name: "ESP-IDF" (Complete)
│   Complete device name; central matches on this
└─ Service UUID: 0xFFFF0 (Complete List of 16-bit Service UUIDs)
    Custom service UUID; central can determine available services during scan
```

2.2 GATT — Generic Attribute Profile

GATT (Generic Attribute Profile) defines the data organization structure of BLE devices.

GATT Custom Service Architecture for This Project

```
Service: 0xFFFF0 (Custom Primary Service)
├─ Characteristic: 0xFFFF1
│   Properties: READ | NOTIFY (Readable + Slave notifies master actively)
│   Direction: Slave → Master
│   Purpose: Slave pushes data to central via Notify every 2 seconds
└─ Characteristic: 0xFFFF2
    Properties: WRITE | WRITE_NR (Supports both acknowledged and unacknowledged writes)
    Direction: Master → Slave
    Purpose: Receives data from central, processed in onwrite() callback
```

2.3 Notify vs Write

Operation	Initiator	Target	Acknowledgment	Typical Scenario
Notify	Slave → Master	0xFFFF1	No ACK	Sensor data reporting, heartbeat counting
Write	Master → Slave	0xFFFF2	WRITE: Requires link layer ACK; WRITE_NR: No ACK	Control command delivery, parameter configuration
Indicate	Slave → Master	(Not used)	Requires master ACK	Critical data reporting, guaranteed delivery

In this project, the slave sends a fixed message via Notify on 0xFFFF1 every 2 seconds; the master can write arbitrary data to 0xFFFF2 at any time, and the slave receives and prints it immediately in `onwrite()`. Both operations are asynchronous and independent, non-blocking.

2.4 NimBLE-Arduino Library Architecture

ESP32-S3 uses **Apache NimBLE** as the BLE Host protocol stack.

Core Class Hierarchy

NimBLEDevice	← Static class, stack initialization + global control
└ NimBLEServer	← GATT server, manages service list
└ NimBLEService	← GATT service, contains multiple characteristics
└ NimBLECharacteristic	← GATT characteristic, data carrier + callback binding
└ NimBLEServerCallbacks	← User overrides onConnect/onDisconnect
└ NimBLEAdvertising	← Advertising parameter config and control
└ NimBLECharacteristicCallbacks	← User overrides onWrite/onRead

Compared to ESP-IDF native NimBLE's callback-style API (`gap_event_cb`, `gatt_svr_access_cb`), the Arduino wrapper maps each event type to **independent C++ virtual methods** implemented through class inheritance. This makes code logic more focused and eliminates the need for `switch/case` dispatching in a single callback function.

2.5 Notify vs Indicate Implementation Differences in Arduino

In Arduino NimBLEDevice library, the difference between notify and indicate is determined by **characteristic properties** and **send methods** together.

Operation	Property Declaration	Send Method	Master Acknowledgment
Notify	<code>NIMBLE_PROPERTY::NOTIFY</code>	<code>pChar->notify()</code>	No
Indicate	<code>NIMBLE_PROPERTY::INDICATE</code>	<code>pChar->indicate()</code>	Yes

```
// Notify – No ACK, suitable for high-frequency sensor data
pCharNotify = pService->createCharacteristic(
    CHAR_NOTIFY_UUID,
    NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::NOTIFY // Declare NOTIFY property
);
pCharNotify->setValue("Hello");
pCharNotify->notify(); // Send ATT Handle Value Notification

// Indicate – Requires ACK, suitable for critical control commands
// pChar->indicate(); // Send ATT Handle Value Indication
```

3. Hardware Dependencies

3.1 ESP32-S3 BLE Hardware Specifications

Specification	Parameter
Wireless Standard	BLE 5.0
Max TX Power	+20 dBm
Receiver Sensitivity	-97 dBm (1Mbps mode)
Max MTU	251 bytes (Auto-negotiated by NimBLE-Arduino)
Antenna	On-board PCB antenna

Specification	Parameter
Advertising Interval	Protocol stack default (~100ms)
Protocol Stack	NimBLE (Apache open source) via NimBLE-Arduino
Arduino TX Power Setting	<code>NimBLEDevice::setPower(ESP_PWR_LVL_P3)</code> — Set to +3dBm in this tutorial

BLE and Wi-Fi share the 2.4GHz RF front-end and can coexist through time-division multiplexing.

3.2 Required Libraries

Library	Purpose	Include Method
<code>NimBLEDevice.h</code>	NimBLE BLE protocol stack Arduino wrapper	<code>#include <NimBLEDevice.h></code>

`NimBLEDevice` library needs to be installed via Arduino IDE library manager: **Tools** → **Manage Libraries** → **Search "NimBLE"** → **Install "NimBLE-Arduino"** (Author: h2zero). Do not confuse with ESP32's built-in "BLE" library (based on Bluedroid).

4. Project Architecture and Flow

File Structure (`[BLE_Slave.ino]`

(`../../..../Arduino/3.Network%20Communication/BLE_Slave/BLE_Slave.ino`), ~90 lines total):

BLE_Slave.ino	
└ #include + Macros (DEVICE_NAME, SERVICE_UUID, CHAR*_UUID)	
└ Global variables (pServer, pCharNotify, g_connected)	
└ class writeCallbacks	← Receives master writes (5.2)
└ class ServerCallbacks	← Connect/disconnect events (5.3)
└ void sendTask()	← FreeRTOS periodic push (5.4)
└ void setup()	← BLE slave initialization six-step method (5.5)
└ void loop()	← Empty loop (5.6)

Program Flow:

setup()	
└ NimBLEDevice::init("ESP-IDF")	// Initialize stack
└ createServer() + setCallbacks(serverCb)	// Create GATT server
└ createService(0xFFFF0)	
└ └ createCharacteristic(0xFFFF1, READ NOTIFY)	// Notify characteristic
└ └ createCharacteristic(0xFFFF2, WRITE WRITE_NR)	// write characteristic + bind
callback	
└ service.start()	
└ startAdvertising()	// Start advertising
sendTask (FreeRTOS):	
└ Every 2s: setValue + notify	// Push data

```
writeCallbacks::onwrite():  
  └─ Print data received from master
```

5. Source Code Analysis

Source code: [Source code](#) → [Arduino](#) → [3.Network Communication](#) → [BLE_Slave](#) → [BLE_Slave.ino](#)

5.1 Header Files and Global Variables

The top of the file contains `#include`, macro definitions, and global variables. `NimBLEDevice.h` is the only header file, which internally aggregates all C++ wrapper classes for the NimBLE protocol stack. Macros use `NIMBLEUUID((uint16_t)0x...)` to wrap 16-bit UUIDs into BLE standard 128-bit UUIDs. Three global variables — server pointer, Notify characteristic pointer, and connection status flag — persist throughout the program lifecycle.

```
// NimBLE BLE stack  
#include <NimBLEDevice.h>  
  
// Advertising device name  
#define DEVICE_NAME      "ESP-IDF"  
// Primary service UUID  
#define SERVICE_UUID     NIMBLEUUID((uint16_t)0xFFFF0)  
// Notify char – slave→central  
#define CHAR_NOTIFY_UUID NIMBLEUUID((uint16_t)0xFFFF1)  
// Write char – central→slave  
#define CHAR_WRITE_UUID  NIMBLEUUID((uint16_t)0xFFFF2)  
  
// BLE GATT server pointer  
NimBLEServer* pServer = nullptr;  
// Notify char pointer  
NimBLECharacteristic* pCharNotify = nullptr;  
// BLE connection status flag  
bool g_connected = false;
```

5.2 Receiving Master Data — WriteCallbacks

`writeCallbacks` is responsible for receiving data sent by the master through the Write characteristic (0xFFFF2). It inherits from `NimBLECharacteristicCallbacks` and overrides `onwrite()`, which NimBLE protocol stack automatically calls each time a master write completes. `getValue()` returns complete data (`std::string`), no manual mbuf operation or packet aggregation timer needed.

```
// Callback class for receiving central write data
class WriteCallbacks : public NimBLECharacteristicCallbacks {
    void onWrite(NimBLECharacteristic* pChar, NimBLEConnInfo& connInfo) override {
        // Get data central just wrote
        std::string recvdData = pChar->getValue();
        Serial.println("=====");
        Serial.printf("Host Message: %s\n", recvdData.c_str());
        Serial.println("=====");
    }
}
WriteCb; // Global instance, bound when creating characteristic
```

5.3 Connect/Disconnect Event Callbacks — ServerCallbacks

Connect and disconnect are two critical events in the BLE slave lifecycle. Inherit from `NimBLEServerCallbacks` and override `onConnect` and `onDisconnect`: on connect, set the status flag for the send task; on disconnect, clear the flag first then call `startAdvertising()` to resume advertising — otherwise the device will be "silent" and the master can never scan it again.

```
// BLE connect/disconnect callbacks class
class ServerCallbacks : public NimBLEServerCallbacks {
    void onConnect(NimBLEServer* pServer, NimBLEConnInfo& connInfo) override {
        // Set connection flag
        g_connected = true;
        Serial.println("Bluetooth connected");
    }

    void onDisconnect(NimBLEServer* pServer, NimBLEConnInfo& connInfo, int reason) override {
        // Clear connection flag
        g_connected = false;
        Serial.println("Bluetooth disconnected");
        // Key: restart advertising for re-discovery
        NimBLEDevice::startAdvertising();
    }
}
ServerCb; // Global instance
```

5.4 FreeRTOS Push Task — sendTask

`sendTask` is an independent FreeRTOS task that checks connection status every 2 seconds in an infinite loop. If the master is online, it pushes data via the Notify characteristic (0xFF1). Core logic is only two lines: `setValue()` updates the local characteristic value cache, `notify()` triggers ATT Handle Value Notification to the master. When not connected, `g_connected` is false and the task silently skips.

```
// FreeRTOS periodic send task
void sendTask(void* param) {
    while (1) {
        // Send only when connected and char is valid
        if (g_connected && pCharNotify) {
            // Set characteristic value
            pCharNotify->setValue("Hello from ESP32 Slave");
            // Send notification to central
        }
    }
}
```

```

    pCharNotify->notify();
    Serial.println("Sent: Hello from ESP32 Slave");
}
// Block 2 seconds
vTaskDelay(pdMS_TO_TICKS(2000));
}
}

```

5.5 setup() — Initialize BLE Slave

`setup()` is Arduino's entry point where all one-time initialization is done. The core flow is divided into six steps — initialize stack → create GATT server → create service → create characteristics → start service → start advertising — finally create FreeRTOS task to start periodic sending.

```

void setup() {
    // Init serial + wait for USB enumeration
    Serial.begin(115200);
    delay(1500);
    Serial.println("\n BLE Slave Started!");

    // Init BLE stack + set advertising name
    NimBLEDevice::init(DEVICE_NAME);
    // TX power +3dBm
    NimBLEDevice::setPower(ESP_PWR_LVL_P3);

    // Create GATT server
    pServer = NimBLEDevice::createServer();
    // Register connect/disconnect callbacks
    pServer->setCallbacks(&serverCb);

    // Create custom GATT service
    NimBLEService* pService = pServer->createService(SERVICE_UUID);

    // Create Notify char – slave→central
    pCharNotify = pService->createCharacteristic(
        CHAR_NOTIFY_UUID,
        NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::NOTIFY
    );

    // Create write char + chained callback binding
    pService->createCharacteristic(
        CHAR_WRITE_UUID,
        NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR
    )->setCallbacks(&writeCb);

    // Register service to ATT attribute table
    pService->start();

    // Configure advertising and start
    NimBLEAdvertising* pAdvertising = NimBLEDevice::getAdvertising();
    // Include service UUID in advertising
    pAdvertising->addServiceUUID(SERVICE_UUID);
}

```



```
// Include device name in advertising
pAdvertising->setName(DEVICE_NAME);
// Start advertising (ch 37/38/39)
pAdvertising->start();

Serial.println("Waiting for host connection...");

// Create FreeRTOS periodic send task
xTaskCreate(sendTask, "sendTask", 2048, NULL, 1, NULL);
}
```

BLE Slave Initialization Six-Step Method

Step	API	Description
1. Initialize Stack	<code>NimBLEDevice::init(DEVICE_NAME)</code>	One line completes what takes 5 steps in ESP-IDF; parameter also serves as advertising device name
2. Create Server	<code>NimBLEDevice::createServer()</code>	Create <code>NimBLEServer</code> instance, manages GATT attribute table
3. Create Service	<code>pServer->createService(SERVICE_UUID)</code>	Create service entry for 0xFFFF0 in ATT table
4. Create Characteristics	<code>pService->createCharacteristic(...)</code>	Add Notify and Write characteristics to service
5. Start Service	<code>pService->start()</code>	Register service + characteristics to ATT; master can discover via GATT
6. Start Advertising	<code>pAdvertising->start()</code>	Periodically send advertising packets on BLE channels 37/38/39

5.6 loop() — Empty Loop

`loop()` is completely empty in BLE applications. All work is distributed across three independent execution flows; Arduino's main task does not need to intervene.

```
void loop() {}
```

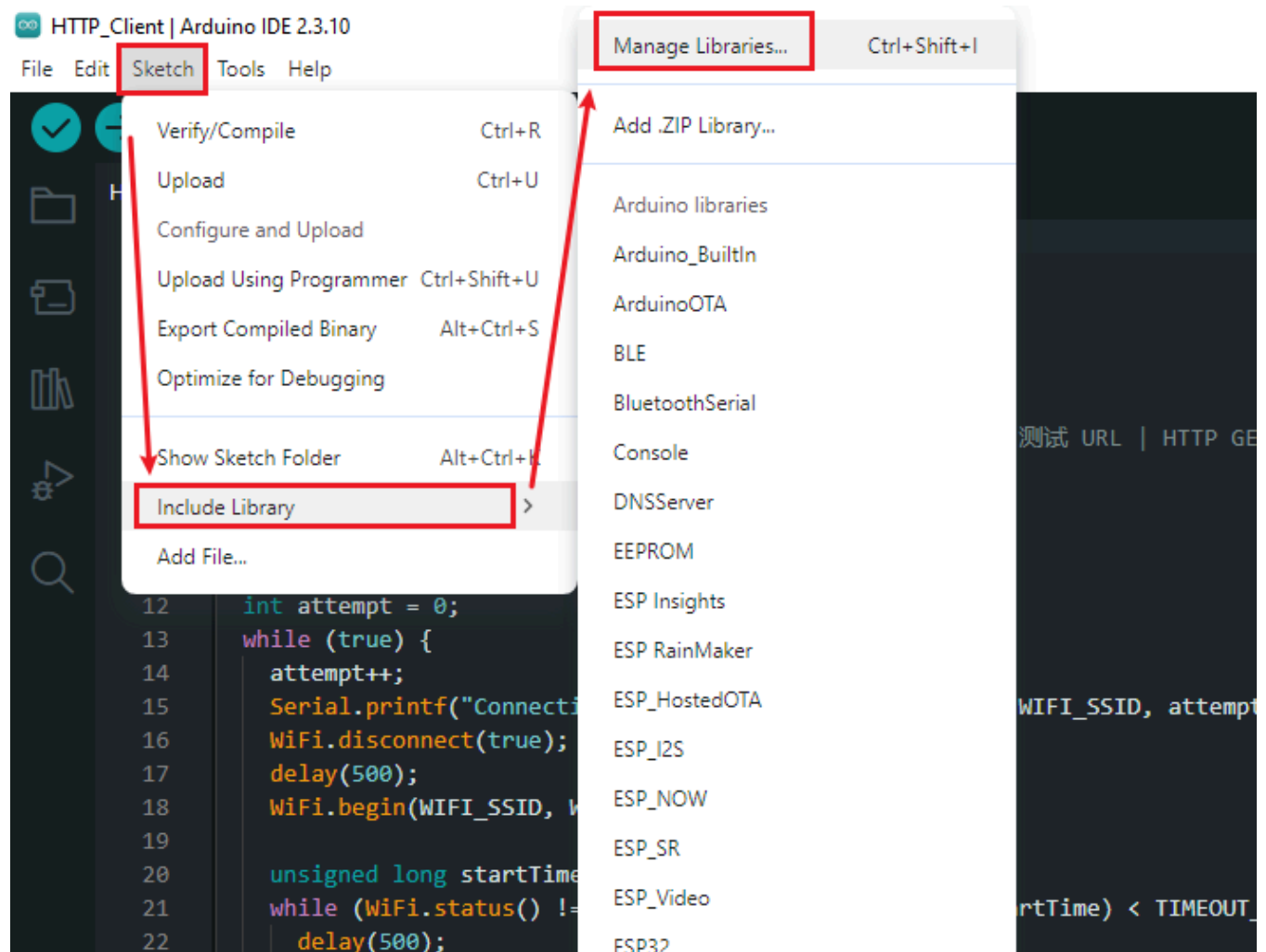
Execution Flow	Responsible For	Running Location
NimBLE Host Task	Protocol stack event loop, handles GAP advertising/connection/disconnection, dispatches GATT write callbacks	Auto-created inside <code>NimBLEDevice::init()</code>
<code>sendTask</code>	Push data via Notify every 2 seconds	User <code>xTaskCreate</code> created
<code>loop()</code>	No operation — placeholder only	Arduino main task

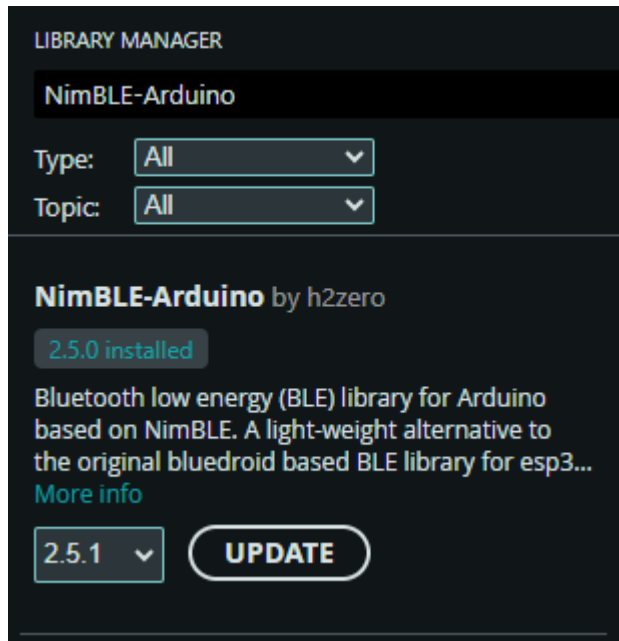
6. Operation Procedures

There are two ways to test the slave: choose one. **Method 1 (Phone Bluetooth Debug)** is suitable when you only have one development board; **Method 2 (Dual-Board Communication)** requires two ESP32-S3 boards with Slave and Master firmware respectively.

6.1 Build and Flash

Prerequisites: NimBLE-Arduino library installed (Tools → Manage Libraries → Search "NimBLE-Arduino" → Install, author h2zero).





Arduino flashing process: See `Arduino` → 1. `Before Development` → 2. `Build and Flash`.

6.2 Method 1: Phone Bluetooth Debug

Applicable Scenario: Only one ESP32-S3 board, using a phone app as BLE master for quick verification.

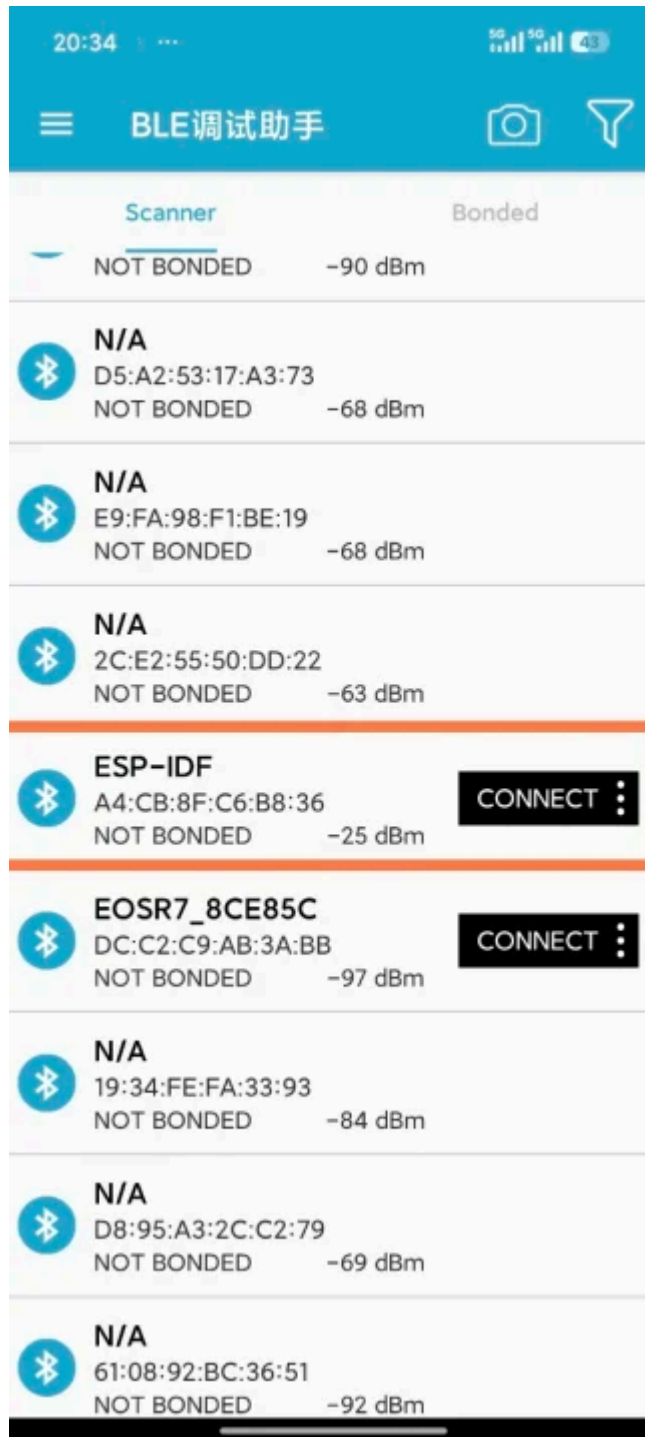
Recommended Apps: **nRF Connect for Mobile** (most feature-rich), LightBlue, EFR Connect, BLE Scanner — all free.

Operation Steps:

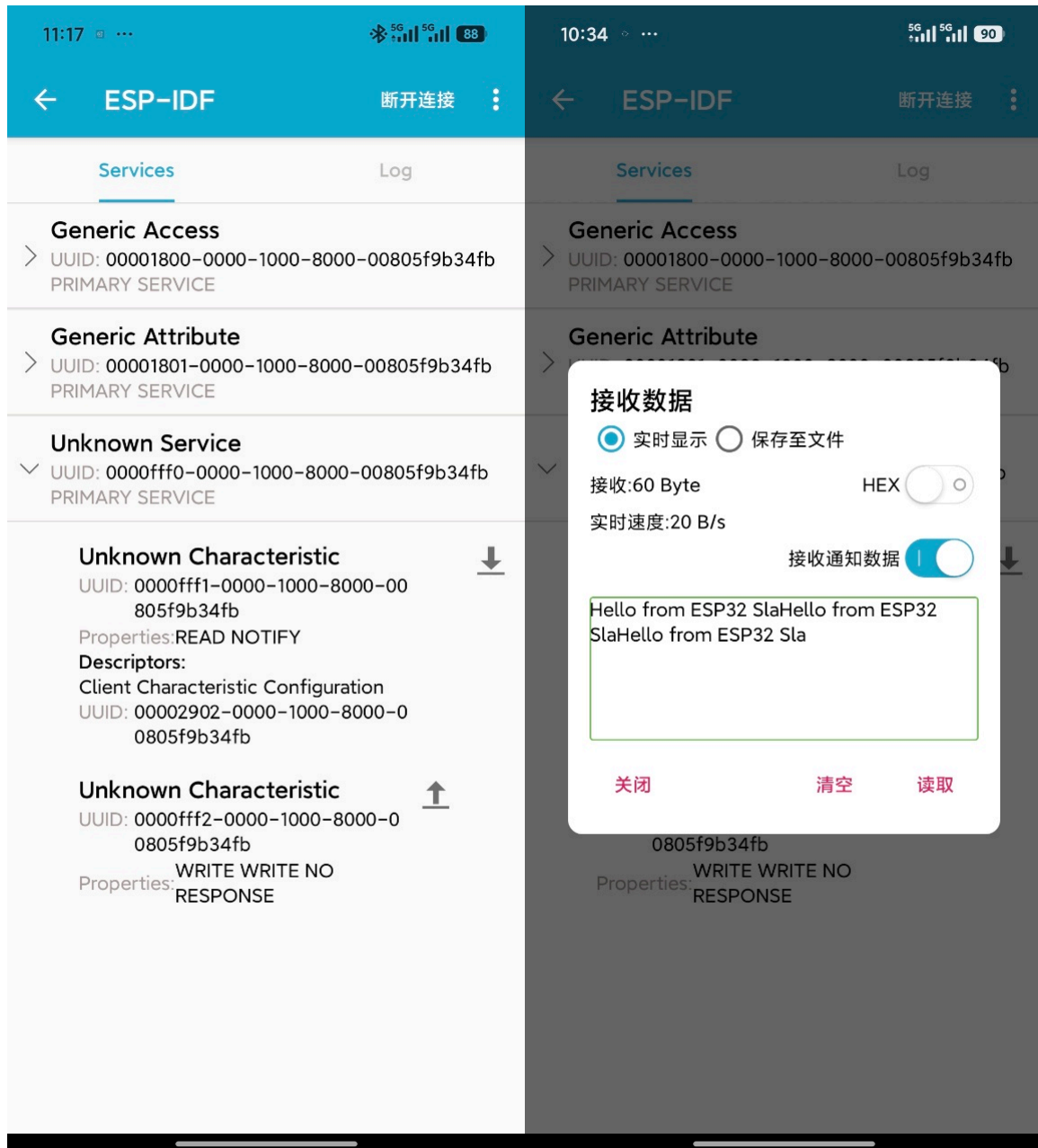
Step	Operation	Expected Result
1	Power on slave	Serial outputs <code>waiting for host connection...</code>
2	Phone app scans	Discovers device <code>ESP-IDF</code>
3	Tap connect	Slave serial outputs <code>Bluetooth connected</code>
4	Find <code>0xFFFF1</code> , enable notifications (Subscribe)	Receives <code>Hello from ESP32 slave</code> push every 2 seconds
5	Find <code>0xFFFF2</code> , write arbitrary text	Slave serial prints <code>Host Message: <text></code>
6	Disconnect Bluetooth	Slave serial outputs <code>Bluetooth disconnected</code> , automatically resumes advertising

nRF Connect Detailed Operation Guide:

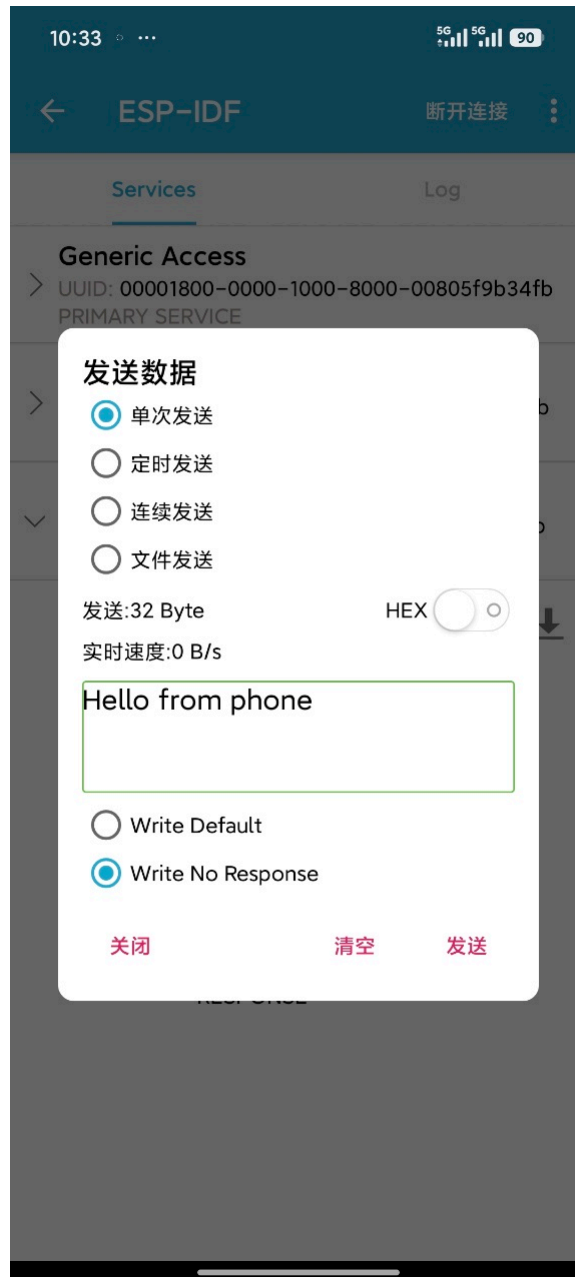
- Scan for Devices:** Open the app, tap `Scan`, find `ESP-IDF` in the device list. Note: If not found, pull down to refresh or move closer to the slave and retry.
- Connect:** Tap `Connect` next to `ESP-IDF`. After successful connection, app top shows `CONNECTED`, bottom tab switches to `CLIENT` page, slave serial prints `Bluetooth connected` synchronously.



3. **Discover Services:** After connection, app automatically performs GATT service discovery, `CLIENT` page lists all services. Find `Unknown Service` with UUID `0xFF0`, tap to expand.
4. **Enable Notifications (Subscribe):** After expanding, you see two characteristics:
 1. `Unknown Characteristic` — UUID `0xFF1`, properties show `READ`, `NOTIFY`
 2. `Unknown Characteristic` — UUID `0xFF2`, properties show `WRITE`, `WRITE NO RESPONSE`
5. Tap the multiple downward arrows icon next to **0xFF1** (`Enable Notifications`). After this, the app automatically pops up `Hello from ESP32 Slave` every 2 seconds.



6. **Write Data:** Tap the upward arrow icon next to **0xFF2** (write). A text input dialog appears. Select **Text** format, enter any text (e.g., **Hello from Phone**), tap **send**. Slave serial immediately prints **Host Message: Hello from Phone**.



7. **Disconnect:** Tap the top `DISCONNECT` button to disconnect. Slave serial prints `Bluetooth disconnected`, while automatically resuming advertising (at this point, immediately tap `Scan` again, `ESP-IDF` will reappear in the device list).

```
Sent: Hello from ESP32 Slave
Bluetooth disconnected
```

LightBlue users: Similar operation — Scan → Connect → Select service `0xFFE0` → For `0xFFE1` click `Listen for notifications` → For `0xFFE2` click `write new value`.

6.3 Method 2: Dual-Board Communication Test (Requires Two Boards)

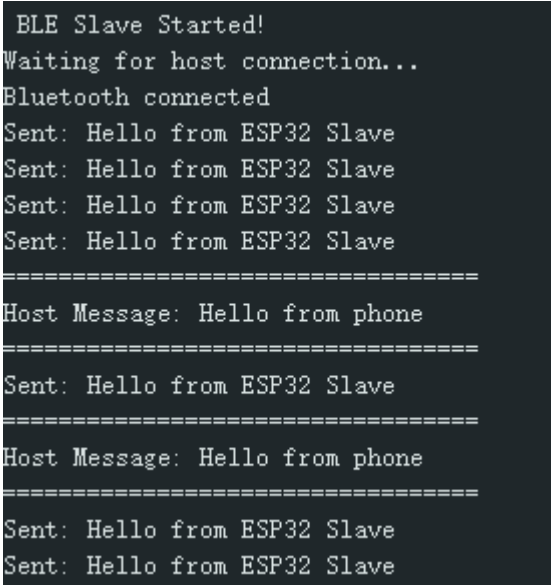
Applicable Scenario: Two ESP32-S3 boards, one as BLE Slave, one as BLE Master, achieving complete dual-device communication.

Step	Operation	Expected Result
1	Power on Slave board	Serial outputs <code>waiting for host connection...</code>
2	Power on Master board	Master scans and auto-connects to <code>ESP-IDF</code>
3	Wait for connection	Slave serial outputs <code>Bluetooth connected</code>
4	Wait ~2 seconds	Slave serial outputs <code>Sent: Hello from ESP32 Slave</code>
5	Master writes to 0xFFFF2	Slave serial prints <code>Host Message: <content></code>
6	Disconnect Master or reset Slave	Serial outputs <code>Bluetooth disconnected</code>

6.4 Expected Output — Phone Debug

Slave Serial:

```
BLE Slave Started!
waiting for host connection...
Bluetooth connected
Sent: Hello from ESP32 Slave
=====
Host Message: Hello from Phone
=====
Sent: Hello from ESP32 Slave
...
Bluetooth disconnected
```



Phone App Side:

Operation	App Display
Scan	Device list shows <code>ESP-IDF</code> , service UUID <code>0xFFFF0</code>

Operation	App Display
After connection	Shows two characteristics: 0xFFFF1 (NOTIFY, READ) and 0xFFFF2 (WRITE, WRITE_NR)
Enable 0xFFFF1 notifications	Pops up Hello from ESP32 Slave every 2 seconds
Write Hello from Phone to 0xFFFF2	Write successful, Slave serial prints synchronously

6.5 Expected Output — Dual-Board Communication

Slave Serial:

```
BLE Slave Started!
Waiting for host connection...
Bluetooth connected
Sent: Hello from ESP32 Slave
Sent: Hello from ESP32 Slave
=====
Host Message: Hello from ESP32 Master
=====
Sent: Hello from ESP32 Slave
...
Bluetooth disconnected
```

```
Sent: Hello from ESP32 Slave
Sent: Hello from ESP32 Slave
=====
Host Message: Hello from ESP32 Master
=====
Sent: Hello from ESP32 Slave
Sent: Hello from ESP32 Slave
Sent: Hello from ESP32 Slave
=====
Host Message: Hello from ESP32 Master
=====
```

Output Rhythm Difference: Slave's `sendTask` pushes at fixed 2-second intervals and prints immediately, so serial log rhythm is strictly 2 seconds per `Sent:`. Master's notification reception goes through NimBLE protocol stack's internal event queue, and Master's own periodic task (usually 5-6 seconds) retrieves data from the queue for printing in batches. So Slave prints two or three times before Master prints once in a batch. **The frequency mismatch only reflects the difference in their respective print cycles and does not affect BLE transmission itself — every single notification is delivered at the NimBLE underlying layer.**

This tutorial only covers Slave-side code. For Master-side implementation and output, please refer to the corresponding **BLE Master Tutorial**.

6.6 Log Field Description

Log	Meaning
<code>BLE Slave started!</code>	Program started, initialization complete
<code>waiting for host connection...</code>	Advertising started, waiting for master scan/connection
<code>Bluetooth connected</code>	Master successfully connected
<code>Bluetooth disconnected</code>	Master disconnected, slave automatically re-advertising
<code>Sent: Hello from ESP32 Slave</code>	Slave pushed Notify via 0xFF1
<code>Host Message: <content></code>	Slave received master's write via 0xFF2

6.7 Edge Cases

Scenario	Behavior
Slave runs standalone (not connected)	Continues advertising, <code>sendTask()</code> silently skips, no data sent
Master disconnects	Clears connection flag, calls <code>startAdvertising()</code> to automatically resume advertising
Master rescans after disconnect	Slave has already resumed advertising, can be rediscovered and connected
Right after connection	<code>sendTask</code> has 2-second cycle, first Notify delayed by at most 2 seconds
Master rapid consecutive writes	Each write is processed independently in <code>onwrite()</code> , no interference

7. FAQ

Symptom	Cause	Solution
Compilation error: <code>NimBLEDevice.h</code> not found	NimBLE-Arduino library not installed, or wrong BLE library installed (Bluedroid)	Library manager search "NimBLE-Arduino" (author h2zero) to install; confirm board selected as <code>ESP32S3 Dev Module</code> ; restart IDE
No serial output	USB CDC On Boot not enabled, or baud rate mismatch	Tools → USB CDC On Boot → Enabled; serial monitor set to 115200; press RST to reset

Symptom	Cause	Solution
Master cannot scan slave	Advertising not started, already connected by another device, too far or interference	Confirm serial has <code>waiting for host connection...</code> ; confirm not taken by another connection; move within 5 meters, away from 2.4GHz interference sources; restart phone Bluetooth
Disconnects immediately after connection	Incompatible connection parameters or <code>onConnect()</code> takes too long	Normal — <code>onDisconnect()</code> auto re-advertises; if persists, check distance and interference; <code>onConnect()</code> in this tutorial does not timeout
Phone app cannot receive Notify data	Notifications not enabled for 0xFFFF1 (CCCD not written 0x0001)	nRF Connect tap downward arrows next to 0xFFFF1 (Enable Notifications); LightBlue enable "Listen for notifications"
Cannot distinguish <code>pAdvertising->start()</code> vs <code>startAdvertising()</code>	Former is called after initial parameter config; latter reuses existing config on disconnect	Use <code>pAdvertising->start()</code> for initial advertising in <code>setup()</code> ; use <code>NimBLEDevice::startAdvertising()</code> for re-advertising in <code>onDisconnect()</code>
How to modify notification content or interval?	—	Modify <code>sendTask()</code> : change <code>setValue()</code> content, adjust <code>vTaskDelay()</code> milliseconds. Note power consumption and channel congestion for high frequency
Slave/Master output frequency inconsistent	Both ends' printing driven by their own task cycles, Slave 2s, Master 5-6s	Normal — does not affect BLE transmission, every notification is delivered at NimBLE underlying layer
Can Wi-Fi and BLE be used simultaneously?	ESP32-S3 supports time-division multiplexing, Arduino core and NimBLE auto-coordinate	Just <code>#include <WiFi.h></code> and <code><NimBLEDevice.h></code> together; note throughput decreases for both, audio streaming not recommended with coexistence